

MYSQL STORED PROCEDURES

Examples and Practice Exercises for MySQL Workbench

1. Setup: Employees Table

```
CREATE DATABASE IF NOT EXISTS employeedb1;  
USE employeedb1;
```

```
CREATE TABLE IF NOT EXISTS employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    dept_id INT,  
    salary DECIMAL(10,2)  
);
```

```
INSERT INTO employees VALUES  
(101,'Rahul',1,50000),  
(102,'Priya',2,65000),  
(103,'Anil',1,55000),  
(104,'Sneha',3,70000),  
(105,'Kiran',2,48000);
```

Query 1

Limit to 1000 rows

```

1 • CREATE DATABASE IF NOT EXISTS employeedb1;
2 • USE employeedb1;
3 • CREATE TABLE IF NOT EXISTS employees (
4     emp_id INT PRIMARY KEY,
5     emp_name VARCHAR(50),
6     dept_id INT,
7     salary DECIMAL(10,2)
8 );
9 • INSERT INTO employees VALUES
10 (101,'Rahul',1,50000),
11 (102,'Priya',2,65000),
12 (103,'Anil',1,55000),
13 (104,'Sneha',3,70000),
14 (105,'Kiran',2,48000);

```

SQLAdditions

Automatic context help is disabled. Use the current caret position to

Context Help

Snippets

Output

Action Output

#	Time	Action	Message
1	12:05:01	CREATE DATABASE IF NOT EXISTS employeedb1	1 row(s) affected, 1 warning(s): 1007 Can't
2	12:05:01	USE employeedb1	0 row(s) affected
3	12:05:01	CREATE TABLE IF NOT EXISTS employees (emp_id INT PRIMARY KEY, emp_...	0 row(s) affected
4	12:05:01	INSERT INTO employees VALUES (101,'Rahul',1,50000), (102,'Priya',2,65000), (103,'...	5 row(s) affected Records: 5 Duplicates:

2. Verify Data

Question: Display all employee records.

```
SELECT * FROM employees;
```

The screenshot shows a database IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The SQL editor contains the following code:

```
9 • INSERT INTO employees VALUES
10 (101, 'Rahul', 1, 50000),
11 (102, 'Priya', 2, 65000),
12 (103, 'Anil', 1, 55000),
13 (104, 'Sneha', 3, 70000),
14 (105, 'Kiran', 2, 48000);
15 • SELECT * FROM employees;
```

Below the editor is the 'Result Grid' tab, which displays a table with 5 rows and 4 columns: emp_id, emp_name, dept_id, and salary.

	emp_id	emp_name	dept_id	salary
▶	101	Rahul	1	50000.00
	102	Priya	2	65000.00
	103	Anil	1	55000.00
	104	Sneha	3	70000.00
	105	Kiran	2	48000.00
*	NULL	NULL	NULL	NULL

To the right of the table is a 'Form Editor' icon. Below the table are 'Apply' and 'Revert' buttons. The 'Context Help' and 'Snippets' tabs are also visible.

The 'Output' pane at the bottom shows the 'Action Output' log:

#	Time	Action	Message
⚠ 1	12:05:01	CREATE DATABASE IF NOT EXISTS employeeedb1	1 row(s) affected, 1 warning(s): 1007 Can't
✓ 2	12:05:01	USE employeeedb1	0 row(s) affected
✓ 3	12:05:01	CREATE TABLE IF NOT EXISTS employees (emp_id INT PRIMARY KEY, emp...	0 row(s) affected
✓ 4	12:05:01	INSERT INTO employees VALUES (101,'Rahul',1,50000), (102,'Priya',2,65000), (103,...	5 row(s) affected Records: 5 Duplicates: 0
✓ 5	12:05:43	SELECT * FROM employees LIMIT 0, 1000	5 row(s) returned

Procedure 1 – Display All Employees

Question: Create a procedure that displays all employees.

```
DROP PROCEDURE IF EXISTS GetEmployees;
DELIMITER $$
```

```
CREATE PROCEDURE GetEmployees()
BEGIN
    SELECT * FROM employees;
END $$
```

```
DELIMITER ;
```

CALL GetEmployees();

The screenshot shows a database IDE interface. At the top, a toolbar includes icons for file operations, execution, and navigation, along with a 'Limit to 1000 rows' dropdown. The main editor displays SQL code for creating and calling a procedure. Below the editor is a 'Result Grid' showing employee data. To the right, a message states 'Automatic context help is disabled. Use the current caret position on'. At the bottom, an 'Output' pane shows a log of actions and their results.

```
3 • CREATE PROCEDURE GetEmployees()
4 BEGIN
5     SELECT * FROM employees;
6 END $$
7 DELIMITER ;
8
9 • CALL GetEmployees();
```

emp_id	emp_name	dept_id	salary
101	Rahul	1	50000.00
102	Priya	2	65000.00
103	Anil	1	55000.00
104	Sneha	3	70000.00
105	Kiran	2	48000.00

#	Time	Action	Message
✓ 7	12:06:47	CREATE PROCEDURE GetEmployees() BEGIN SELECT * FROM employees; E...	0 row(s) affected
✓ 8	12:06:47	CALL GetEmployees()	5 row(s) returned
✗ 9	12:07:21	CALL GetEmployeeById(102)	Error Code: 1305. PROCEDURE employeeedb1.0
✓ 10	12:09:09	DROP PROCEDURE IF EXISTS GetEmployees	0 row(s) affected
✓ 11	12:09:09	CREATE PROCEDURE GetEmployees() BEGIN SELECT * FROM employees; E...	0 row(s) affected
✓ 12	12:09:09	CALL GetEmployees()	5 row(s) returned

Procedure 2 – IN Parameter

Question: Create a procedure that accepts an employee ID and displays that employee's details.

Query 1

Limit to 1000 rows

```

4  BEGIN
5      SELECT * FROM employees
6      WHERE emp_id = eid;
7  END $$
8  DELIMITER ;
9
10 • CALL GetEmployeeById(102);

```

SQLAdditions

Automatic context help is disabled. Use the current caret position o

Result Grid

Filter Rows:

Export:

	emp_id	emp_name	dept_id	salary
▶	102	Priya	2	65000.00

Result Grid

Form Editor

Result 4

Read Only

Context Help

Snippets

Output

Action Output

#	Time	Action	Message
✓ 10	12:09:09	DROP PROCEDURE IF EXISTS GetEmployees	0 row(s) affected
✓ 11	12:09:09	CREATE PROCEDURE GetEmployees() BEGIN SELECT * FROM employees; E...	0 row(s) affected
✓ 12	12:09:09	CALL GetEmployees()	5 row(s) returned
⚠ 13	12:09:33	DROP PROCEDURE IF EXISTS GetEmployeeById	0 row(s) affected, 1 warning(s): 1305 PROCEDURE
✓ 14	12:09:33	CREATE PROCEDURE GetEmployeeById(IN eid INT) BEGIN SELECT * FROM ...	0 row(s) affected
✓ 15	12:09:33	CALL GetEmployeeById(102)	1 row(s) returned

Procedure 3 – Update Salary

Question: Create a procedure that accepts employee ID and amount, then increases the employee's salary.

Query 1 x

SQLAdditions

Limit to 1000 rows

```

9      SET salary = salary + amount
10     WHERE emp_id = eid;
11 END $$
12 DELIMITER ;
13
14 • CALL IncreaseSalary(101,5000);
15 • SELECT * FROM employees WHERE emp_id=101;

```

Automatic context help is disabled. Use the current caret position to search for help.

Result Grid

	emp_id	emp_name	dept_id	salary
▶	101	Rahul	1	55000.00
*	NULL	NULL	NULL	NULL

Filter Rows: Edit: Result Grid Form Editor

employees 5 x Apply Revert Context Help Snippets

Output

Action Output

#	Time	Action	Message
✓ 14	12:09:33	CREATE PROCEDURE GetEmployeeById(IN eid INT) BEGIN SELECT * FROM ...	0 row(s) affected
✓ 15	12:09:33	CALL GetEmployeeById(102)	1 row(s) returned
⚠ 16	12:10:08	DROP PROCEDURE IF EXISTS IncreaseSalary	0 row(s) affected, 1 warning(s): 1305 PROCEDURE
✓ 17	12:10:08	CREATE PROCEDURE IncreaseSalary(IN eid INT, IN amount DECIMAL(10, ...)	0 row(s) affected
✓ 18	12:10:08	CALL IncreaseSalary(101,5000)	1 row(s) affected
✓ 19	12:10:08	SELECT * FROM employees WHERE emp_id=101 LIMIT 0, 1000	1 row(s) returned

Procedure 4 – Aggregate Function

Question: Create a procedure to calculate the average salary of all employees.

```

DROP PROCEDURE IF EXISTS AvgSalary;
DELIMITER $$

```

```

CREATE PROCEDURE AvgSalary()
BEGIN
    SELECT AVG(salary) AS AverageSalary
    FROM employees;
END $$

```

```

DELIMITER ;

```


CALL AvgSalary();

The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' button. The query window contains the following SQL code:

```
4 BEGIN
5     SELECT AVG(salary) AS AverageSalary
6     FROM employees;
7 END $$
8 DELIMITER ;
9
10 CALL AvgSalary();
```

Below the query window, the 'Result Grid' is displayed with the following data:

AverageSalary
58600.000000

On the right side, a message states: 'Automatic context help is disabled. Use the current caret position to view context help for the current statement.'

At the bottom, the 'Output' window shows the 'Action Output' log:

#	Time	Action	Message
✓ 17	12:10:08	CREATE PROCEDURE IncreaseSalary(IN eid INT, IN amount DECIMAL(10,...	0 row(s) affected
✓ 18	12:10:08	CALL IncreaseSalary(101,5000)	1 row(s) affected
✓ 19	12:10:08	SELECT * FROM employees WHERE emp_id=101 LIMIT 0, 1000	1 row(s) returned
⚠ 20	12:10:32	DROP PROCEDURE IF EXISTS AvgSalary	0 row(s) affected, 1 warning(s): 1305 PRO
✓ 21	12:10:32	CREATE PROCEDURE AvgSalary() BEGIN SELECT AVG(salary) AS AverageSa...	0 row(s) affected
✓ 22	12:10:32	CALL AvgSalary()	1 row(s) returned

Procedure 5 – JOIN

Question: Create a procedure to display employee name, department name and salary. Run the department setup first.

```
CREATE TABLE IF NOT EXISTS departments (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50)  
);
```

```
INSERT IGNORE INTO departments VALUES  
(1,'CSE'),(2,'ECE'),(3,'EEE');
```

```
DROP PROCEDURE IF EXISTS EmployeeDepartment;  
DELIMITER $$
```

```
CREATE PROCEDURE EmployeeDepartment()  
BEGIN  
    SELECT e.emp_name, d.dept_name, e.salary  
    FROM employees e  
    JOIN departments d ON e.dept_id=d.dept_id;  
END $$
```

```
DELIMITER ;
```

```
CALL EmployeeDepartment();
```

The screenshot displays a SQL IDE interface. The top window, titled 'Query 1', contains the following SQL code:

```
11 SELECT e.emp_name, d.dept_name, e.salary  
12 FROM employees e  
13 JOIN departments d ON e.dept_id=d.dept_id;  
14 END $$  
15 DELIMITER ;  
16  
17 CALL EmployeeDepartment();
```

Below the query editor is the 'Result Grid' showing the output of the query:

	emp_name	dept_name	salary
▶	Rahul	CSE	55000.00
	Priya	ECE	65000.00
	Anil	CSE	55000.00
	Sneha	EEE	70000.00
	Kiran	ECE	48000.00

At the bottom, the 'Output' window shows the execution log:

#	Time	Action	Message
✓ 22	12:10:32	CALL AvgSalary()	1 row(s) returned
✓ 23	12:11:00	CREATE TABLE IF NOT EXISTS departments (dept_id INT PRIMARY KEY, ...	0 row(s) affected
✓ 24	12:11:00	INSERT IGNORE INTO departments VALUES(1,'CSE'),(2,'ECE'),(3,'EEE')	3 row(s) affected Records: 3 Duplicates: 0 Wa
⚠ 25	12:11:00	DROP PROCEDURE IF EXISTS EmployeeDepartment	0 row(s) affected, 1 warning(s): 1305 PROCED
✓ 26	12:11:00	CREATE PROCEDURE EmployeeDepartment() BEGIN SELECT e.emp_name, d...	0 row(s) affected
✓ 27	12:11:00	CALL EmployeeDepartment()	5 row(s) returned

Procedure 6 – GROUP BY and AVG

Question: Create a procedure to display department-wise employee count and average salary.

```
DROP PROCEDURE IF EXISTS DepartmentAverageSalary;  
DELIMITER $$
```

```
CREATE PROCEDURE DepartmentAverageSalary()  
BEGIN  
    SELECT d.dept_name,  
           COUNT(e.emp_id) AS EmployeeCount,  
           AVG(e.salary) AS AverageSalary  
    FROM employees e  
    JOIN departments d ON e.dept_id=d.dept_id  
    GROUP BY d.dept_id, d.dept_name;  
END $$
```

```
DELIMITER ;
```

```
CALL DepartmentAverageSalary();
```

Query 1

```

8      FROM employees e
9      JOIN departments d ON e.dept_id=d.dept_id
10     GROUP BY d.dept_id, d.dept_name;
11 END $$
12 DELIMITER ;
13
14 • CALL DepartmentAverageSalary();

```

Result Grid

Filter Rows:

Export:

	dept_name	EmployeeCount	AverageSalary
▶	CSE	2	55000.000000
	ECE	2	56500.000000
	EEE	1	70000.000000

Result Grid

Form Editor

Automatic context help is disabled. Use the current caret position on

Context Help Snippets

Output

Action Output

#	Time	Action	Message
25	12:11:00	DROP PROCEDURE IF EXISTS EmployeeDepartment	0 row(s) affected, 1 warning(s): 1305 PROCED
26	12:11:00	CREATE PROCEDURE EmployeeDepartment() BEGIN SELECT e.emp_name, d...	0 row(s) affected
27	12:11:00	CALL EmployeeDepartment()	5 row(s) returned
28	12:11:34	DROP PROCEDURE IF EXISTS DepartmentAverageSalary	0 row(s) affected, 1 warning(s): 1305 PROCED
29	12:11:34	CREATE PROCEDURE DepartmentAverageSalary() BEGIN SELECT d.dept_na...	0 row(s) affected
30	12:11:34	CALL DepartmentAverageSalary()	3 row(s) returned

Automatic context help is disabled. Use the current caret position on

Procedure 7 – IF...ELSE

```
DROP PROCEDURE IF EXISTS SalaryStatus;
DELIMITER $$
```

```
SELECT salary INTO sal
FROM employees
WHERE emp_id=eid;
```

```

IF sal >= 60000 THEN
    SELECT 'High Salary' AS Status;
ELSE
    SELECT 'Low Salary' AS Status;
END IF;
END $$

```

```

DELIMITER ;

```

```

CALL SalaryStatus(102);

```

The screenshot shows a SQL IDE interface. The script editor on the left contains the following SQL code:

```

12     ELSE
13         SELECT 'Low Salary' AS Status;
14     END IF;
15 END $$
16 DELIMITER ;
17
18 • CALL SalaryStatus(102);

```

Below the script editor is a "Result Grid" showing a single row with the status "High Salary".

On the right side, there is a "Context Help" panel with the text: "Automatic context help is disabled. Use the current caret position to search for help." Below this is a "Snippets" panel.

At the bottom, there is an "Output" window showing the "Action Output" of the script execution:

#	Time	Action	Message
28	12:11:34	DROP PROCEDURE IF EXISTS DepartmentAverageSalary	0 row(s) affected, 1 warning(s): 1305 PROCEDURE does not exist
29	12:11:34	CREATE PROCEDURE DepartmentAverageSalary() BEGIN SELECT d.dept_name, AVG(s.salary) AS avg_salary FROM employees s JOIN departments d ON s.dept_id = d.dept_id GROUP BY d.dept_name;	0 row(s) affected
30	12:11:34	CALL DepartmentAverageSalary()	3 row(s) returned
31	12:12:02	DROP PROCEDURE IF EXISTS SalaryStatus	0 row(s) affected, 1 warning(s): 1305 PROCEDURE does not exist
32	12:12:02	CREATE PROCEDURE SalaryStatus(IN eid INT) BEGIN DECLARE sal DECIMAL(10,2); SELECT s.salary INTO sal FROM employees s WHERE s.emp_id = eid;	0 row(s) affected
33	12:12:02	CALL SalaryStatus(102)	1 row(s) returned

Procedure 8 – OUT Parameter

Question: Return the total number of employees using an OUT parameter.

```
DROP PROCEDURE IF EXISTS EmployeeCount;
DELIMITER $$
```

```
CREATE PROCEDURE EmployeeCount(OUT total INT)
BEGIN
    SELECT COUNT(*) INTO total
    FROM employees;
END $$
```

```
DELIMITER ;
```

```
CALL EmployeeCount(@total);
SELECT @total AS TotalEmployees;
```

The screenshot shows a SQL IDE interface. The main window displays a script with the following SQL code:

```
5 SELECT COUNT(*) INTO total
6 FROM employees;
7 END $$
8 DELIMITER ;
9
10 CALL EmployeeCount(@total);
11 SELECT @total AS TotalEmployees;
```

A tooltip "Save the script to a file." is visible over the script editor. Below the script editor, the "Result Grid" tab is active, showing a single row with the column "TotalEmployees" and the value "5".

To the right of the script editor, a message states: "Automatic context help is disabled. Use the current caret position of the script to search for context help."

At the bottom, the "Output" window is open, showing a table of execution results:

#	Time	Action	Message
32	12:12:02	CREATE PROCEDURE SalaryStatus(IN eid INT) BEGIN DECLARE sal DECIMA...	0 row(s) affected
33	12:12:02	CALL SalaryStatus(102)	1 row(s) returned
34	12:12:28	DROP PROCEDURE IF EXISTS EmployeeCount	0 row(s) affected, 1 warning(s): 1305 PROCEDURE
35	12:12:28	CREATE PROCEDURE EmployeeCount(OUT total INT) BEGIN SELECT COUN...	0 row(s) affected
36	12:12:28	CALL EmployeeCount(@total)	1 row(s) affected
37	12:12:29	SELECT @total AS TotalEmployees LIMIT 0, 1000	1 row(s) returned

Procedure 9 – INOUT Parameter

Question: Receive a bonus through an INOUT parameter and add 5000 to it.

```
DROP PROCEDURE IF EXISTS AddBonus;  
DELIMITER $$
```

```
CREATE PROCEDURE AddBonus(INOUT bonus DECIMAL(10,2))  
BEGIN  
    SET bonus = bonus + 5000;  
END $$
```

```
DELIMITER ;
```

```
SET @bonus=10000;  
CALL AddBonus(@bonus);  
SELECT @bonus AS UpdatedBonus;
```


Query 1 x SQLAdditions

Limit to 1000 rows

```

5      SET bonus = bonus + 5000;
6  END $$
7  DELIMITER ;
8
9  • SET @bonus=10000;
10 CALL AddBonus(@bonus);
11 SELECT @bonus AS UpdatedBonus;

```

Automatic context help is disabled. Use the current caret position to search for help.

Result Grid

UpdatedBonus
15000.00

Filter Rows: Export: Result Grid Form Editor

Result 11 x Read Only Context Help Snippets

Output

Action Output

#	Time	Action	Message
✓ 37	12:12:29	SELECT @total AS TotalEmployees LIMIT 0, 1000	1 row(s) returned
⚠ 38	12:12:59	DROP PROCEDURE IF EXISTS AddBonus	0 row(s) affected, 1 warning(s): 1305 PROCEDURE does not exist
✓ 39	12:12:59	CREATE PROCEDURE AddBonus(INOUT bonus DECIMAL(10,2)) BEGIN SET ...	0 row(s) affected
✓ 40	12:12:59	SET @bonus=10000	0 row(s) affected
✓ 41	12:12:59	CALL AddBonus(@bonus)	0 row(s) affected
✓ 42	12:12:59	SELECT @bonus AS UpdatedBonus LIMIT 0, 1000	1 row(s) returned

Procedure 10 – Transaction

Question: Transfer an amount from one employee salary record to another using START TRANSACTION and COMMIT.

```

DROP PROCEDURE IF EXISTS TransferSalaryAmount;
DELIMITER $$

```

```

CREATE PROCEDURE TransferSalaryAmount(
    IN fromEmp INT,
    IN toEmp INT,
    IN amount DECIMAL(10,2)
)
BEGIN

```

START TRANSACTION;

UPDATE employees
SET salary=salary-amount
WHERE emp_id=fromEmp;

UPDATE employees
SET salary=salary+amount
WHERE emp_id=toEmp;

COMMIT;
END \$\$

DELIMITER ;

CALL TransferSalaryAmount(101,102,2000);

SELECT * FROM employees WHERE emp_id IN (101,102);

The screenshot shows a SQL IDE interface. The top window, titled 'Query 1', contains the following SQL code:

```
18  
19     COMMIT;  
20 END $$  
21 DELIMITER ;  
22  
23 • CALL TransferSalaryAmount(101,102,2000);  
24 • SELECT * FROM employees WHERE emp_id IN (101,102);
```

Below the query window is the 'Result Grid' showing the output of the last query. It has columns for emp_id, emp_name, dept_id, and salary.

	emp_id	emp_name	dept_id	salary
▶	101	Rahul	1	53000.00
	102	Priya	2	67000.00
*	NULL	NULL	NULL	NULL

Below the result grid is the 'Output' window, titled 'employees 12'. It shows a log of database actions and their results.

#	Time	Action	Message
✓ 41	12:12:59	CALL AddBonus(@bonus)	0 row(s) affected
✓ 42	12:12:59	SELECT @bonus AS UpdatedBonus LIMIT 0, 1000	1 row(s) returned
⚠ 43	12:13:52	DROP PROCEDURE IF EXISTS TransferSalaryAmount	0 row(s) affected, 1 warning(s): 1305 PROCEDURE DOES NOT EXIST
✓ 44	12:13:52	CREATE PROCEDURE TransferSalaryAmount(IN fromEmp INT, IN toEmp INT, IN amount INT)	0 row(s) affected
✓ 45	12:13:52	CALL TransferSalaryAmount(101,102,2000)	0 row(s) affected
✓ 46	12:13:52	SELECT * FROM employees WHERE emp_id IN (101,102) LIMIT 0, 1000	2 row(s) returned

4. Useful Procedure Commands

SHOW PROCEDURE STATUS

WHERE Db='employeeedb1';

SHOW CREATE PROCEDURE GetEmployees;

DROP PROCEDURE IF EXISTS GetEmployees;

Query 1 x

Limit to 1000 rows

```

1 • SHOW PROCEDURE STATUS WHERE Db='employeeedb1';
2 • SHOW CREATE PROCEDURE GetEmployees;

```

Result Grid

Procedure	sql_mode	Create Procedure	character_set_client
GetEmployees	ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLE...	CREATE DEFINER='root'@'localhost' PROCE...	utf8mb4

Result 13 Result 14 x

Output

Action Output

#	Time	Action	Message
43	12:13:52	DROP PROCEDURE IF EXISTS TransferSalaryAmount	0 row(s) affected, 1 warning(s): 1305 PROCE
44	12:13:52	CREATE PROCEDURE TransferSalaryAmount(IN fromEmp INT, IN toEmp IN...	0 row(s) affected
45	12:13:52	CALL TransferSalaryAmount(101,102,2000)	0 row(s) affected
46	12:13:52	SELECT * FROM employees WHERE emp_id IN (101,102) LIMIT 0, 1000	2 row(s) returned
47	12:14:25	SHOW PROCEDURE STATUS WHERE Db='employeeedb1'	10 row(s) returned
48	12:14:25	SHOW CREATE PROCEDURE GetEmployees	1 row(s) returned

5. Why DELIMITER is Used

DELIMITER temporarily changes the statement terminator in MySQL Workbench. It prevents the semicolons inside BEGIN...END from being interpreted as the end of the entire CREATE PROCEDURE statement.

```
DELIMITER $$
```

```
CREATE PROCEDURE Example()
```

```
BEGIN
```

```
    SELECT * FROM employees;
```

```
END $$
```

```
DELIMITER ;
```

Limit to 1000 rows

```
5 BEGIN
6     SELECT * FROM employees;
7 END $$
8 DELIMITER ;
9
10 -- Calling the procedure to generate a result for your screenshot
11 CALL Example();
```

Result Grid Filter Rows: Export: Wrap Cell Content:

	emp_id	emp_name	dept_id	salary
▶	101	Rahul	1	53000.00
	102	Priya	2	67000.00
	103	Anil	1	55000.00
	104	Sneha	3	70000.00
	105	Kiran	2	48000.00

Result 15 x

Output

Action Output

#	Time	Action	Message
✓ 46	12:13:52	SELECT * FROM employees WHERE emp_id IN (101,102) LIMIT 0, 1000	2 row(s) returned
✓ 47	12:14:25	SHOW PROCEDURE STATUS WHERE Db='employeeedb1'	10 row(s) returned
✓ 48	12:14:25	SHOW CREATE PROCEDURE GetEmployees	1 row(s) returned
⚠ 49	12:17:28	DROP PROCEDURE IF EXISTS Example	0 row(s) affected, 1 warning(s): 1305 PROCEDURE DOES NOT EXIST
✓ 50	12:17:28	CREATE PROCEDURE Example() BEGIN SELECT * FROM employees; END	0 row(s) affected
✓ 51	12:17:28	-- Calling the procedure to generate a result for your screenshot CALL Example()	5 row(s) returned

6. Student Practice Tasks

1. Create a procedure to display employees whose salary is greater than a given amount.

The screenshot displays the SQL Developer interface. The top toolbar includes icons for file operations, execution, and settings. The main editor window contains the following SQL code:

```
5      SELECT * FROM employees
6      WHERE salary > min_amount;
7  END $$
8  DELIMITER ;
9
10 -- Test the procedure
11 CALL GetEmployeesBySalary(55000);
```

Below the editor, the 'Result Grid' shows the output of the query. It has columns for emp_id, emp_name, dept_id, and salary. Two rows are displayed:

emp_id	emp_name	dept_id	salary
102	Priya	2	67000.00
104	Sneha	3	70000.00

The bottom section of the interface shows the 'Output' window with a tab for 'Action Output'. It contains a log of database actions:

#	Time	Action	Message
49	12:17:28	DROP PROCEDURE IF EXISTS Example	0 row(s) affected, 1 warning(s): 1305 PROC
50	12:17:28	CREATE PROCEDURE Example() BEGIN SELECT * FROM employees; END	0 row(s) affected
51	12:17:28	-- Calling the procedure to generate a result for your screenshot CALL Example()	5 row(s) returned
52	12:19:08	DROP PROCEDURE IF EXISTS GetEmployeesBySalary	0 row(s) affected, 1 warning(s): 1305 PROC
53	12:19:08	CREATE PROCEDURE GetEmployeesBySalary(IN min_amount DECIMAL(10,2)) B...	0 row(s) affected
54	12:19:09	-- Test the procedure CALL GetEmployeesBySalary(55000)	2 row(s) returned

2. Create a procedure to update an employee's department using IN parameters.

```
7      WHERE emp_id = e_id;
8  END $$
9  DELIMITER ;
10
11  -- Test the procedure by moving Anil (103) to dept 2
12  CALL UpdateEmployeeDept(103, 2);
13  SELECT * FROM employees WHERE emp_id = 103;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: [I/A](#)

	emp_id	emp_name	dept_id	salary
▶	103	Anil	2	55000.00
*	NULL	NULL	NULL	NULL

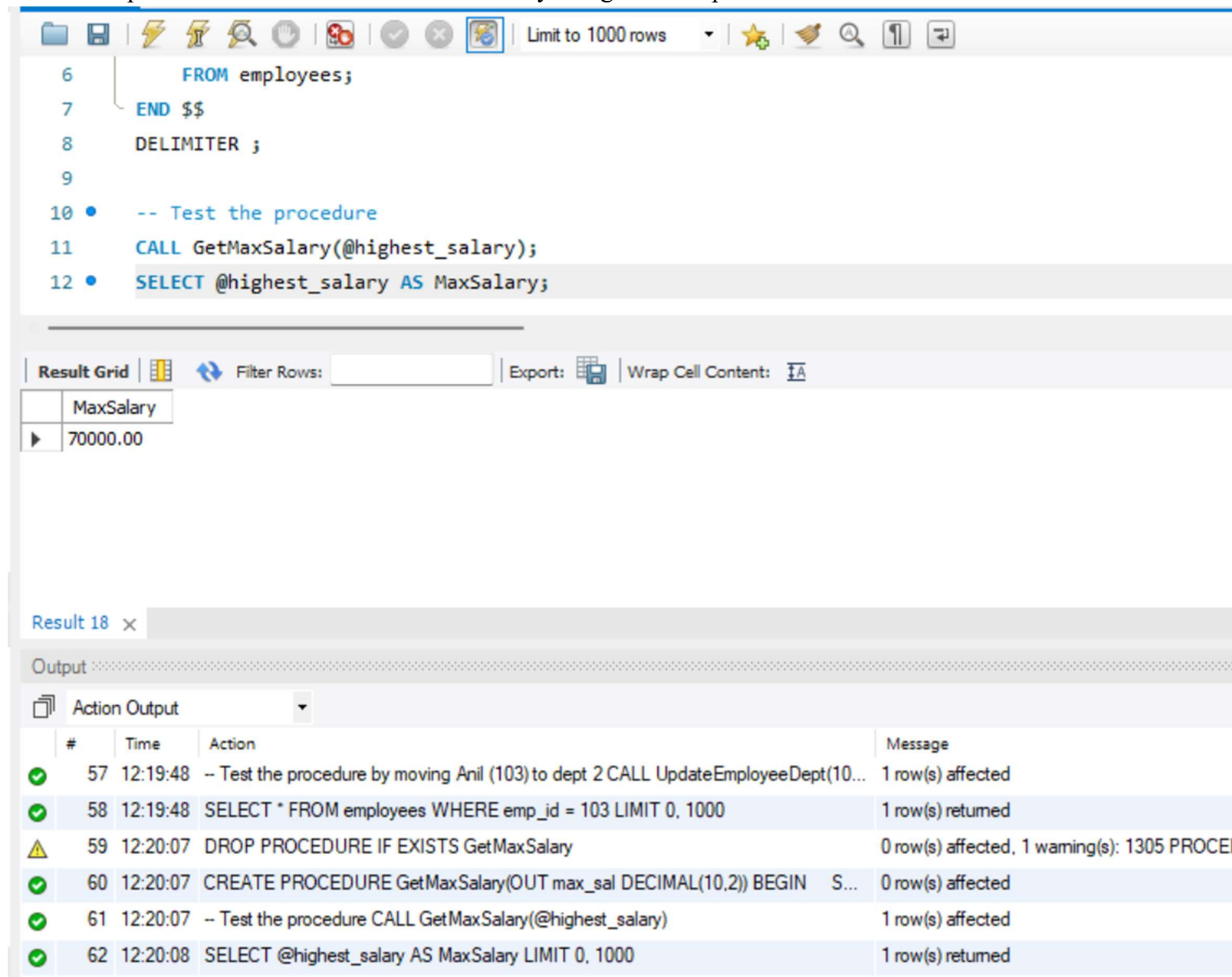
employees 17 x

Output

Action Output

#	Time	Action	Message
✓ 53	12:19:08	CREATE PROCEDURE GetEmployeesBySalary(IN min_amount DECIMAL(10,2)) B...	0 row(s) affected
✓ 54	12:19:09	-- Test the procedure CALL GetEmployeesBySalary(55000)	2 row(s) returned
⚠ 55	12:19:48	DROP PROCEDURE IF EXISTS UpdateEmployeeDept	0 row(s) affected, 1 warning(s): 1305 PROCEDU
✓ 56	12:19:48	CREATE PROCEDURE UpdateEmployeeDept(IN e_id INT, IN new_d_id INT) BEG...	0 row(s) affected
✓ 57	12:19:48	-- Test the procedure by moving Anil (103) to dept 2 CALL UpdateEmployeeDept(10...	1 row(s) affected
✓ 58	12:19:48	SELECT * FROM employees WHERE emp_id = 103 LIMIT 0, 1000	1 row(s) returned

3. Create a procedure to return the maximum salary using an OUT parameter.



```
6      FROM employees;
7  END $$
8  DELIMITER ;
9
10 • -- Test the procedure
11 CALL GetMaxSalary(@highest_salary);
12 • SELECT @highest_salary AS MaxSalary;
```

Result Grid

MaxSalary
70000.00

Result 18 x

Output

Action Output

#	Time	Action	Message
✓ 57	12:19:48	-- Test the procedure by moving Anil (103) to dept 2 CALL UpdateEmployeeDept(10...	1 row(s) affected
✓ 58	12:19:48	SELECT * FROM employees WHERE emp_id = 103 LIMIT 0, 1000	1 row(s) returned
⚠ 59	12:20:07	DROP PROCEDURE IF EXISTS GetMaxSalary	0 row(s) affected, 1 warning(s): 1305 PROCEDURE DOES NOT EXIST
✓ 60	12:20:07	CREATE PROCEDURE GetMaxSalary(OUT max_sal DECIMAL(10,2)) BEGIN S...	0 row(s) affected
✓ 61	12:20:07	-- Test the procedure CALL GetMaxSalary(@highest_salary)	1 row(s) affected
✓ 62	12:20:08	SELECT @highest_salary AS MaxSalary LIMIT 0, 1000	1 row(s) returned

4. Create a procedure to return the employee count of a specified department.

Query 1 x

Limit to 1000 rows

```
6      FROM employees
7      WHERE dept_id = d_id;
8  END $$
9  DELIMITER ;
10
11 • -- Test the procedure for department 2
12 CALL GetDeptEmployeeCount(2);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	EmployeeCount
▶	3

Result 19 x

Output

Action Output

#	Time	Action	Message
✓ 60	12:20:07	CREATE PROCEDURE GetMaxSalary(OUT max_sal DECIMAL(10,2)) BEGIN S...	0 row(s) affected
✓ 61	12:20:07	-- Test the procedure CALL GetMaxSalary(@highest_salary)	1 row(s) affected
✓ 62	12:20:08	SELECT @highest_salary AS MaxSalary LIMIT 0, 1000	1 row(s) returned
⚠ 63	12:20:35	DROP PROCEDURE IF EXISTS GetDeptEmployeeCount	0 row(s) affected, 1 warning(s): 1305 PROCED
✓ 64	12:20:35	CREATE PROCEDURE GetDeptEmployeeCount(IN d_id INT) BEGIN SELECT C...	0 row(s) affected
✓ 65	12:20:35	-- Test the procedure for department 2 CALL GetDeptEmployeeCount(2)	1 row(s) returned

5. Create a procedure using JOIN to display employee and department details.

The screenshot shows a database IDE with a SQL editor at the top and a results pane at the bottom. The SQL editor contains the following code:

```
6      FROM employees e
7      JOIN departments d ON e.dept_id = d.dept_id;
8  END $$
9  DELIMITER ;
10
11  -- Test the procedure
12  CALL EmployeeDeptDetails();
```

The results pane displays a table with 5 rows and 4 columns: emp_id, emp_name, dept_name, and salary.

emp_id	emp_name	dept_name	salary
101	Rahul	CSE	53000.00
102	Priya	ECE	67000.00
103	Anil	ECE	55000.00
104	Sneha	EEE	70000.00
105	Kiran	ECE	48000.00

Below the table, the 'Output' pane shows the 'Action Output' for the execution of the SQL script. It lists 6 actions with their status, time, and message.

#	Time	Action	Message
63	12:20:35	DROP PROCEDURE IF EXISTS GetDeptEmployeeCount	0 row(s) affected, 1 warning(s): 1305 PROCEDURE DOES NOT EXIST
64	12:20:35	CREATE PROCEDURE GetDeptEmployeeCount(IN d_id INT) BEGIN SELECT C...	0 row(s) affected
65	12:20:35	-- Test the procedure for department 2 CALL GetDeptEmployeeCount(2)	1 row(s) returned
66	12:20:57	DROP PROCEDURE IF EXISTS EmployeeDeptDetails	0 row(s) affected, 1 warning(s): 1305 PROCEDURE DOES NOT EXIST
67	12:20:57	CREATE PROCEDURE EmployeeDeptDetails() BEGIN SELECT e.emp_id, e.em...	0 row(s) affected
68	12:20:57	-- Test the procedure CALL EmployeeDeptDetails()	5 row(s) returned

6. Create a procedure to display departments whose average salary is greater than 55000.

```

8      GROUP BY d.dept_id, d.dept_name
9      HAVING AVG(e.salary) > 55000;
10  END $$
11  DELIMITER ;
12
13  -- Test the procedure
14  CALL HighAvgSalaryDepts();

```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	dept_name	AverageSalary			
▶	ECE	56666.666667			
	EEE	70000.000000			

Result 21 ✕

Output

Action Output

#	Time	Action	Message
	66 12:20:57	DROP PROCEDURE IF EXISTS EmployeeDeptDetails	0 row(s) affected, 1 warning(s): 1305 PROCED
	67 12:20:57	CREATE PROCEDURE EmployeeDeptDetails() BEGIN SELECT e.emp_id, e.em...	0 row(s) affected
	68 12:20:57	-- Test the procedure CALL EmployeeDeptDetails()	5 row(s) returned
	69 12:21:19	DROP PROCEDURE IF EXISTS HighAvgSalaryDepts	0 row(s) affected, 1 warning(s): 1305 PROCED
	70 12:21:19	CREATE PROCEDURE HighAvgSalaryDepts() BEGIN SELECT d.dept_name, A...	0 row(s) affected
	71 12:21:19	-- Test the procedure CALL HighAvgSalaryDepts()	2 row(s) returned

7. Create a procedure using IF...ELSE to classify salary as High, Medium or Low.

```

16      SELECT 'Low Salary' AS Status;
17  END IF;
18  END $$
19  DELIMITER ;
20
21  • -- Test the procedure for employee 101
22  CALL ClassifySalary(101);

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	Status
▶	Low Salary

Result 22 x

Output

Action Output

	#	Time	Action	Message
	69	12:21:19	DROP PROCEDURE IF EXISTS HighAvgSalaryDepts	0 row(s) affected, 1 warning(s): 1305 PROCED
	70	12:21:19	CREATE PROCEDURE HighAvgSalaryDepts() BEGIN SELECT d.dept_name, A...	0 row(s) affected
	71	12:21:19	-- Test the procedure CALL HighAvgSalaryDepts()	2 row(s) returned
	72	12:21:50	DROP PROCEDURE IF EXISTS ClassifySalary	0 row(s) affected, 1 warning(s): 1305 PROCED
	73	12:21:50	CREATE PROCEDURE ClassifySalary(IN eid INT) BEGIN DECLARE sal DECIM...	0 row(s) affected
	74	12:21:50	-- Test the procedure for employee 101 CALL ClassifySalary(101)	1 row(s) returned

8. Create a procedure that accepts a department ID and returns its average salary.

```
6      FROM employees
7      WHERE dept_id = d_id;
8  END $$
9  DELIMITER ;
10
11 • -- Test the procedure for department 1
12 CALL GetAvgSalaryByDept(1);
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	AverageSalary			
▶	53000.000000			

Result 23 x				
Output				
Action Output				
#	Time	Action	Message	
72	12:21:50	DROP PROCEDURE IF EXISTS ClassifySalary	0 row(s) affected, 1 warning(s): 1305 PROCED	
73	12:21:50	CREATE PROCEDURE ClassifySalary(IN eid INT) BEGIN DECLARE sal DECIM...	0 row(s) affected	
74	12:21:50	-- Test the procedure for employee 101 CALL ClassifySalary(101)	1 row(s) returned	
75	12:22:18	DROP PROCEDURE IF EXISTS GetAvgSalaryByDept	0 row(s) affected, 1 warning(s): 1305 PROCED	
76	12:22:18	CREATE PROCEDURE GetAvgSalaryByDept(IN d_id INT) BEGIN SELECT AV...	0 row(s) affected	
77	12:22:18	-- Test the procedure for department 1 CALL GetAvgSalaryByDept(1)	1 row(s) returned	

9. Create a procedure using INOUT to apply a bonus to an input salary.

Query 1 x

Limit to 1000 rows

```
7  END $$
8  DELIMITER ;
9
10 • -- Test the procedure
11  SET @my_salary = 50000;
12 • CALL ApplyBonus(@my_salary);
13 • SELECT @my_salary AS SalaryWithBonus;
```

Result Grid

	SalaryWithBonus
▶	52000.00

Result 24 x

Output

Action Output

#	Time	Action	Message
✓ 77	12:22:18	-- Test the procedure for department 1 CALL GetAvgSalaryByDept(1)	1 row(s) returned
⚠ 78	12:22:57	DROP PROCEDURE IF EXISTS ApplyBonus	0 row(s) affected, 1 warning(s): 1305 PROCED
✓ 79	12:22:57	CREATE PROCEDURE ApplyBonus(INOUT current_salary DECIMAL(10,2)) BEGI...	0 row(s) affected
✓ 80	12:22:57	-- Test the procedure SET @my_salary = 50000	0 row(s) affected
✓ 81	12:22:57	CALL ApplyBonus(@my_salary)	0 row(s) affected
✓ 82	12:22:57	SELECT @my_salary AS SalaryWithBonus LIMIT 0, 1000	1 row(s) returned

10. Create a procedure that performs two related updates inside a transaction.

The screenshot displays a database IDE interface. The top section contains a SQL script with the following lines:

```
20 COMMIT;
21 END $$
22 DELIMITER ;
23
24 • -- Test the procedure by giving 1000 to employees 101 and 104
25 CALL TwoUpdatesTransaction(101, 104, 1000);
26 • SELECT * FROM employees WHERE emp_id IN (101, 104);
```

Below the script is the 'Result Grid' section, which shows the output of the last query. It contains a table with the following data:

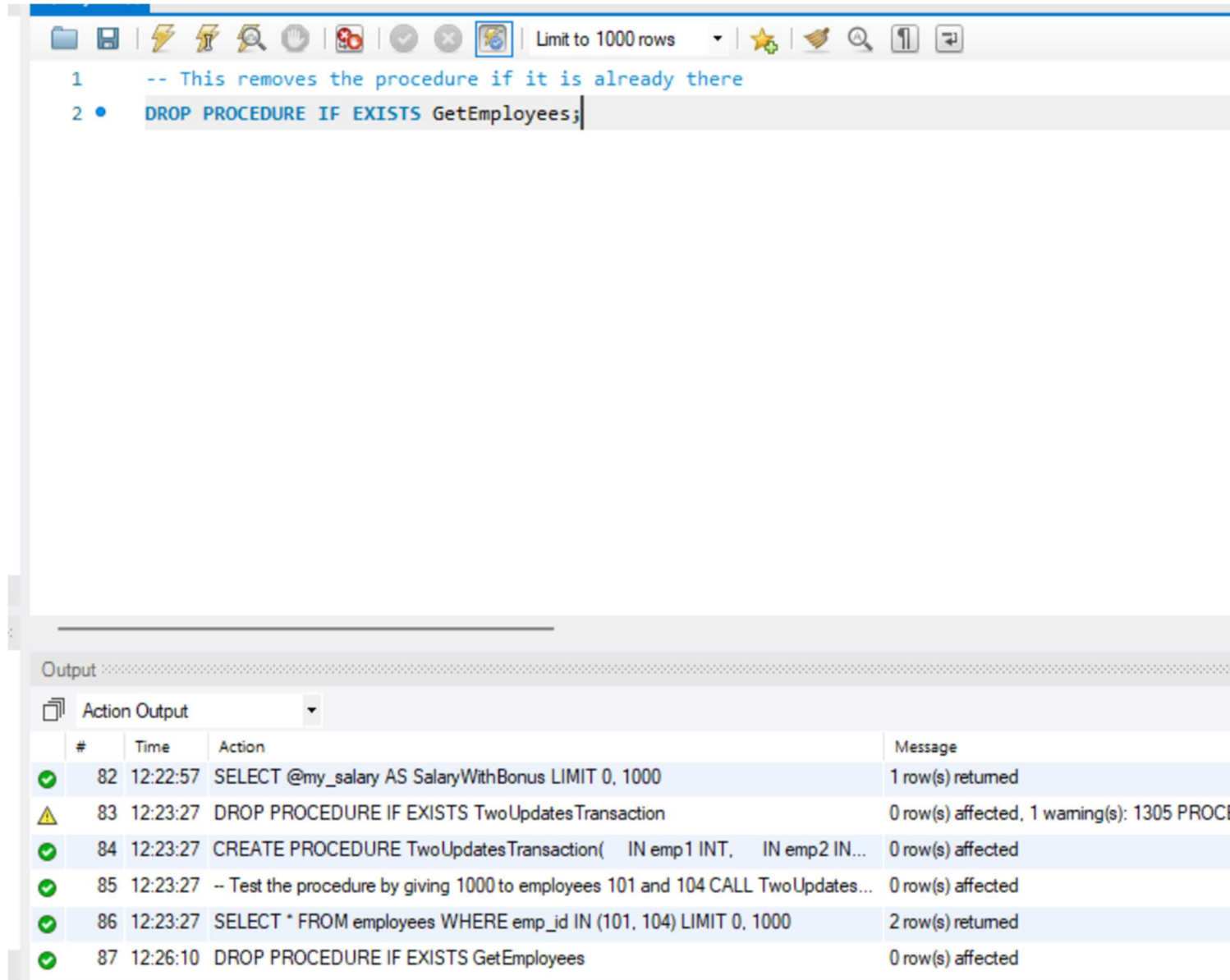
emp_id	emp_name	dept_id	salary
101	Rahul	1	54000.00
104	Sneha	3	71000.00
*	NULL	NULL	NULL

At the bottom of the IDE is the 'Output' section, which shows the execution log. It includes a table with the following data:

#	Time	Action	Message
81	12:22:57	CALL ApplyBonus(@my_salary)	0 row(s) affected
82	12:22:57	SELECT @my_salary AS SalaryWithBonus LIMIT 0, 1000	1 row(s) returned
83	12:23:27	DROP PROCEDURE IF EXISTS TwoUpdatesTransaction	0 row(s) affected, 1 warning(s): 1305 PROCEDURE does not exist
84	12:23:27	CREATE PROCEDURE TwoUpdatesTransaction(IN emp1 INT, IN emp2 INT, IN bonus INT)	0 row(s) affected
85	12:23:27	-- Test the procedure by giving 1000 to employees 101 and 104 CALL TwoUpdatesTransaction(101, 104, 1000);	0 row(s) affected
86	12:23:27	SELECT * FROM employees WHERE emp_id IN (101, 104) LIMIT 0, 1000	2 row(s) returned

7. Common Errors

Procedure already exists: Use DROP PROCEDURE IF EXISTS before recreating it.



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and search. The script editor contains two lines of SQL code:

```
1 -- This removes the procedure if it is already there
2 • DROP PROCEDURE IF EXISTS GetEmployees;
```

Below the script editor is the 'Output' window, which is currently displaying the 'Action Output' tab. It contains a table with the following data:

	#	Time	Action	Message
✓	82	12:22:57	SELECT @my_salary AS SalaryWithBonus LIMIT 0, 1000	1 row(s) returned
⚠	83	12:23:27	DROP PROCEDURE IF EXISTS TwoUpdatesTransaction	0 row(s) affected, 1 warning(s): 1305 PROC
✓	84	12:23:27	CREATE PROCEDURE TwoUpdatesTransaction(IN emp1 INT, IN emp2 IN...	0 row(s) affected
✓	85	12:23:27	-- Test the procedure by giving 1000 to employees 101 and 104 CALL TwoUpdates...	0 row(s) affected
✓	86	12:23:27	SELECT * FROM employees WHERE emp_id IN (101, 104) LIMIT 0, 1000	2 row(s) returned
✓	87	12:26:10	DROP PROCEDURE IF EXISTS GetEmployees	0 row(s) affected

Syntax error near BEGIN: Check DELIMITER \$\$ and DELIMITER ;.

The screenshot shows the SQL Server Enterprise Manager interface. At the top, there's a toolbar with various icons. Below it, a query window titled "Query 1" contains the following T-SQL code:

```

3 • CREATE PROCEDURE TestSyntax()
4 BEGIN
5     SELECT 'No syntax errors here!' AS Message;
6 END $$
7 DELIMITER ;
8
9 • CALL TestSyntax();

```

Below the query window, the "Result Grid" tab is active, displaying a single row of results:

Message
No syntax errors here!

At the bottom, the "Output" pane shows the execution log under the "Action Output" tab:

#	Time	Action	Message
✓ 84	12:23:27	CREATE PROCEDURE TwoUpdatesTransaction(IN emp1 INT, IN emp2 IN...	0 row(s) affected
✓ 85	12:23:27	-- Test the procedure by giving 1000 to employees 101 and 104 CALL TwoUpdates...	0 row(s) affected
✓ 86	12:23:27	SELECT * FROM employees WHERE emp_id IN (101, 104) LIMIT 0, 1000	2 row(s) returned
✓ 87	12:26:10	DROP PROCEDURE IF EXISTS GetEmployees	0 row(s) affected
✓ 88	12:26:51	CREATE PROCEDURE TestSyntax() BEGIN SELECT 'No syntax errors here!' A...	0 row(s) affected
✓ 89	12:26:51	CALL TestSyntax()	1 row(s) returned

Table does not exist: Run USE employeeedb1; and SHOW TABLES;

Query 1 x

Limit to 1000 rows

```
1  -- This selects your database and proves the table exists
2  •  USE employeeedb1;
3  SHOW TABLES;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: IA

Tables_in_employeeedb1
▶ departments
employee
employees

Result 27 x

Output

Action Output

#	Time	Action	Message
✓ 86	12:23:27	SELECT * FROM employees WHERE emp_id IN (101, 104) LIMIT 0, 1000	2 row(s) returned
✓ 87	12:26:10	DROP PROCEDURE IF EXISTS GetEmployees	0 row(s) affected
✓ 88	12:26:51	CREATE PROCEDURE TestSyntax() BEGIN SELECT 'No syntax errors here!' A...	0 row(s) affected
✓ 89	12:26:51	CALL TestSyntax()	1 row(s) returned
✓ 90	12:27:33	USE employeeedb1	0 row(s) affected
✓ 91	12:27:33	SHOW TABLES	3 row(s) returned

Unknown column: Check names using DESCRIBE employees;

The screenshot shows a SQL IDE interface. At the top, a toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. Below the toolbar, a script editor contains two lines of SQL code:

```
1  -- This shows you the exact names and data types of all your columns
2  DESCRIBE employees;
```

Below the script editor, a 'Result Grid' tab is active. It displays the schema of the 'employees' table with the following columns: Field, Type, Null, Key, Default, and Extra.

Field	Type	Null	Key	Default	Extra
emp_id	int	NO	PRI	NULL	
emp_name	varchar(50)	YES		NULL	
dept_id	int	YES		NULL	
salary	decimal(10,2)	YES		NULL	

Below the result grid, an 'Output' tab is active, showing a log of database actions. The log includes a table with columns: #, Time, Action, and Message.

#	Time	Action	Message
87	12:26:10	DROP PROCEDURE IF EXISTS GetEmployees	0 row(s) affected
88	12:26:51	CREATE PROCEDURE TestSyntax() BEGIN SELECT 'No syntax errors here!' A...	0 row(s) affected
89	12:26:51	CALL TestSyntax()	1 row(s) returned
90	12:27:33	USE employeeedb1	0 row(s) affected
91	12:27:33	SHOW TABLES	3 row(s) returned
92	12:28:00	DESCRIBE employees	4 row(s) returned